Excalidraw agent brief

01 / Product and boundaries

HOW TO USE: Copy the prompt on pages 1-4 and attach this PDF so the agent can see the mockups on pages 5-10. The same prompt and seven original PNGs are also embedded as PDF attachments.

PROMPT START

Act as a product designer and system architect. Create an editable Excalidraw planning board for **Elevateo Mail**. Explain it to a beginner in plain English. Use the attached Fable frontend mockups as the visual starting point. Improve workflows where useful, and explain each change beside its sketch.

The product we are building

A personal inbox and opted-in email campaign app for **10 completely separate users**, each with up to **10 connected mailboxes**. Google accounts may come from different organisations or personal Gmail. Users can connect mailboxes, add recipients and import historical messages.

Limit: 30 outgoing emails per connected mailbox per day. This includes manual, campaign and AI sends through the app. Aliases and reconnecting must not create fresh allowances. With 10 mailboxes, the ceiling is 300 per user and 3,000 across the pilot each day. Earlier 600-per-user wishes are superseded. Rolling 24-hour accounting and counting every To/CC/BCC recipient are proposals awaiting approval. External sending is outside the app's control.

Support a web app and directly downloadable Windows and macOS apps, with their own window, icon and email notifications. Approved, fully prepared campaigns must continue in the cloud when the computer is off. Local CLI AI cannot run on a switched-off computer.

Users bring their own Codex/Claude accounts and provider keys. AI may read selected mailboxes and prepare drafts by default; sending is enabled and approved separately. Hunter and AMF are optional **enrichment**; AI handles **personalisation**. AMF's exact product/API is still unidentified.

Be precise about what exists

Fable's Ledger frontend is an interactive prototype using fictional data and simulated sends. Its backend, real provider connections, AI bridge, DNS broker and desktop installers do not exist yet. Label every board section **Prototype**, **Proposed** or **Needs decision**. Screenshot counts are sample data, not capacity evidence. Backend standards still require approval.

Target hosting below **A$15/month**, excluding email, AI, enrichment, signing and external approval costs. Show cost drivers and limits; do not promise this budget without measured usage. Keep CPU, memory and background work low. Correctness and privacy come before clever optimisation.

Draw the system

02 / Board sections 1-4

1. The whole app at a glance

Draw separate areas for the user's device, our cloud and outside providers. Proposed stack: shared React/TypeScript UI; Tauri 2 desktop wrapper; one modular Cloudflare Workers backend. D1 stores structured records, R2 stores bodies/files, Queues carries background work and a scheduled Worker finds due work. Explain each box in one line. Draw arrows labelled with the actual data or action.

Show identity and permission checks at every user-data boundary. Each user owns their connections, messages, campaigns, files, keys and approvals. Use server-side ownership checks on every request and object access. Draw one backend with clear internal responsibilities, not a fleet of microservices.

2. Receiving mail and keeping the inbox current

Gmail/Workspace: Google Pub/Sub change hint -> cloud sync job -> Gmail history/API -> saved message metadata -> UI and eligible device notification. Show watch renewal, missed-event recovery, expired login and visible last-sync time. Fetch bodies and attachments when needed. Treat email HTML and attachments as untrusted. Never render arbitrary email HTML directly into the app.

Resend is proposed for eligible opted-in campaigns from owned, verified domains. It does not replace the connected personal inbox or let users send as arbitrary @gmail.com addresses. Route replies to a connected mailbox. Show authenticated, deduplicated provider events for delivery, bounce and complaints. Start self-hosted support with one tested, reachable HTTPS/JMAP provider; IMAP/SMTP needs its own compatibility and cost decision.

3. Sending safely, including when the device is off

Draw: draft -> recipient/content approval -> durable send record and pending-job record saved together -> queue -> final checks -> reserve mailbox allowance atomically -> provider -> recorded result. Final checks include owner, sending permission, approved version, consent, unsubscribes/suppression, campaign state and provider limits. Edited recipients or content invalidate prior approval.

The database is the record of truth; queue jobs may repeat. Show duplicate protection, fair per-user scheduling, backoff and failed work. A timeout may mean the provider accepted the email: label it **Unknown outcome**, hold reserved quota and investigate before retrying. Never promise exactly-once delivery. Pause blocks new dispatches; handed-off mail may complete. Unsubscribes and replies stop relevant future campaign work according to policy.

4. AI and DNS: separate, limited access

Desktop UI -> local bridge -> Codex CLI -> permission-checked tool/MCP gateway -> selected mailboxes. Initially Claude fallback can only draft with tools disabled. Keep mailbox context disabled until provider data-use requirements and CLI isolation are verified. No browser-to-CLI bridge initially. One job at a time, cancellation, timeout and process cleanup. CLIs can send data to cloud services; Tauri permissions do not sandbox them. Email text never grants tool authority. Encrypt credentials; keep them out of prompts/logs. No silent permission escalation or fallback after an uncertain send.

Draw DNS as a separate broker: exact approved subdomain/record -> before/after preview -> human approval -> audited change. Mail and AI workers receive no DNS credentials. Keep company root MX unchanged on boltloop.co, flooda.co and elevateoco.com; do not repoint existing in.elevateoco.com. Gmail API access needs no MX change. Provider zone-wide tokens do not prove subdomain isolation: keep writes manual until containment is proven.

Sketch the daily workflows

03 / Board sections 5-6

5. Bring Fable's frontend into the board

Embed all seven reference screenshots in labelled reference frames. Next to them, draw editable wireframes using Excalidraw shapes and text. Keep the calm Ledger style: light neutral surfaces, strong type, indigo accents, clear spacing, a compact sidebar and fast keyboard-friendly actions. Preserve the dark and mobile references. Use diagrams to explain behaviour, not just appearance.

R1 Overview (page 5): urgent work and failures first, then useful figures. Every figure opens the matching filtered list. **R2 Inbox (page 6):** mailbox/folder scope, visible filters, account identity on each row, bulk actions and undo. **R3 Split reader (page 7):** keep list context, read and reply without losing filters; make the sender account obvious.

R4 Campaigns (page 8): status, progress, replies, review and pause. **R5/R6 Mobile (page 9):** one pane at a time with easy back navigation and full-screen compose. **R7 Dark (page 10):** preserve contrast, focus and meaning without depending on colour.

6. Fill the missing product screens

Connections: Mailboxes, Sending, AI, Enrichment and Domains. Show capabilities, permission scope, last sync, remaining daily allowance, reconnect and revoke. Keys go into a secure connection form, never chat. Include account login/MFA and the Google connection flow as proposed screens; do not imply external Google approval is already complete.

AI assistant: selected-mailbox chips, exact context being shared, CLI status, stop button, editable draft and a separate sending approval. The app must still work when AI or enrichment is unavailable. Enrichment failure must not block inbox use.

Campaign builder: recipients -> consent checks -> message/personalisation -> schedule and mailbox limits -> final review -> activate. Explain what will run in the cloud, what is waiting on local AI and what pause can still stop. Optional Instantly/Smartlead support is a future adapter with only one scheduler controlling any campaign.

Imports and recovery: connect another mailbox, paste/import recipients, import old messages with preview, duplicates and errors. Show bounded progress and cancel. A lost connection or stale inbox must not look like an empty inbox. Include loading, empty, offline, permission denied, quota reached, failed and unknown-send states.

Domain review and distribution: draw an exact-subdomain DNS preview with root records visibly locked. Sketch a small download page with Open web app, Windows installer (.exe) and Mac download (.dmg), plus update status. Signing, macOS notarisation and platform release checks are planned work, not finished features.

Connect screens to real responsibilities

Beside each main screen add three short notes: **What the user does / What the backend must provide / What happens if it fails.** Sketch success and the most likely failure state. Label proposed data such as sender account, last sync, quota remaining, approval version and send status so the future frontend and backend agree. Never label delivery as proof that someone read the email.

Make it useful to build from

04 / Board sections 7-8 and handoff

7. Efficiency, health and the build map

Explain the low-cost choices: indexed, paginated lists; metadata first; lazy bodies/files; push-led sync with bounded recovery; small queue messages; batched imports; one on-demand CLI process. Keep long imports from starving inbox/send work. Queues already supplies background messaging: no Redis, Kafka, RabbitMQ or search cluster without measured need.

Propose a small health screen for stale sync, queue delay, unknown/failed sends, revoked access, mailbox allowance, storage and backup age. Use bounded, redacted logs and an independent outage check. Show backup restoration and key recovery as required operational work. Do not build a full monitoring platform or assume a dashboard can detect its own outage.

Draw the proposed repository map: apps/web, apps/desktop, apps/backend; packages/domain, contracts, providers and shared UI only when genuinely shared; migrations, infra, tests, docs and .planning. Explain each folder in a few words. Current frontend lives in src/; this is a future migration proposal, not a claim those folders exist. Avoid empty scaffolding.

Show the few tests that protect the product: cross-user access denial, send/quota races, duplicate jobs, timeout reconciliation, consent/unsubscribe, pause, expired credentials, backup restore and critical desktop/mobile journeys. Use Docker, fake providers and existing Vitest/Playwright tooling for routine tests with no paid API calls. Real authorised provider checks and Windows/macOS release checks are still needed before launch.

8. Decisions and the smallest delivery sequence

Create a short decision list: quota window and recipient counting; history/import/storage limits; login/MFA and data region; exact AMF and first self-hosted provider; external Gmail approval and permitted AI data use; DNS containment; backup/key recovery; signing owner and cost. Identify the consequence of each choice. Ten users is not proof of exemption from Google requirements.

Draw stages: approve standards/provider requirements -> one mailbox -> two isolated users with reliable sync/send/recovery -> PC-off campaigns -> desktop/local AI -> 10-user load/cost checks -> optional providers. Ask targeted questions; mark unresolved decisions and continue independent diagrams.

Required files and quality checks

Deliver elevateo-mail-planning.excalidraw with editable boxes, text and connectors, all seven screenshots embedded, and a clear reading order across the eight sections. Also export a readable PDF and one PNG per main section. Keep the original screenshot reference frames and new wireframes separate; do not flatten the whole board into one image or rely on local image paths.

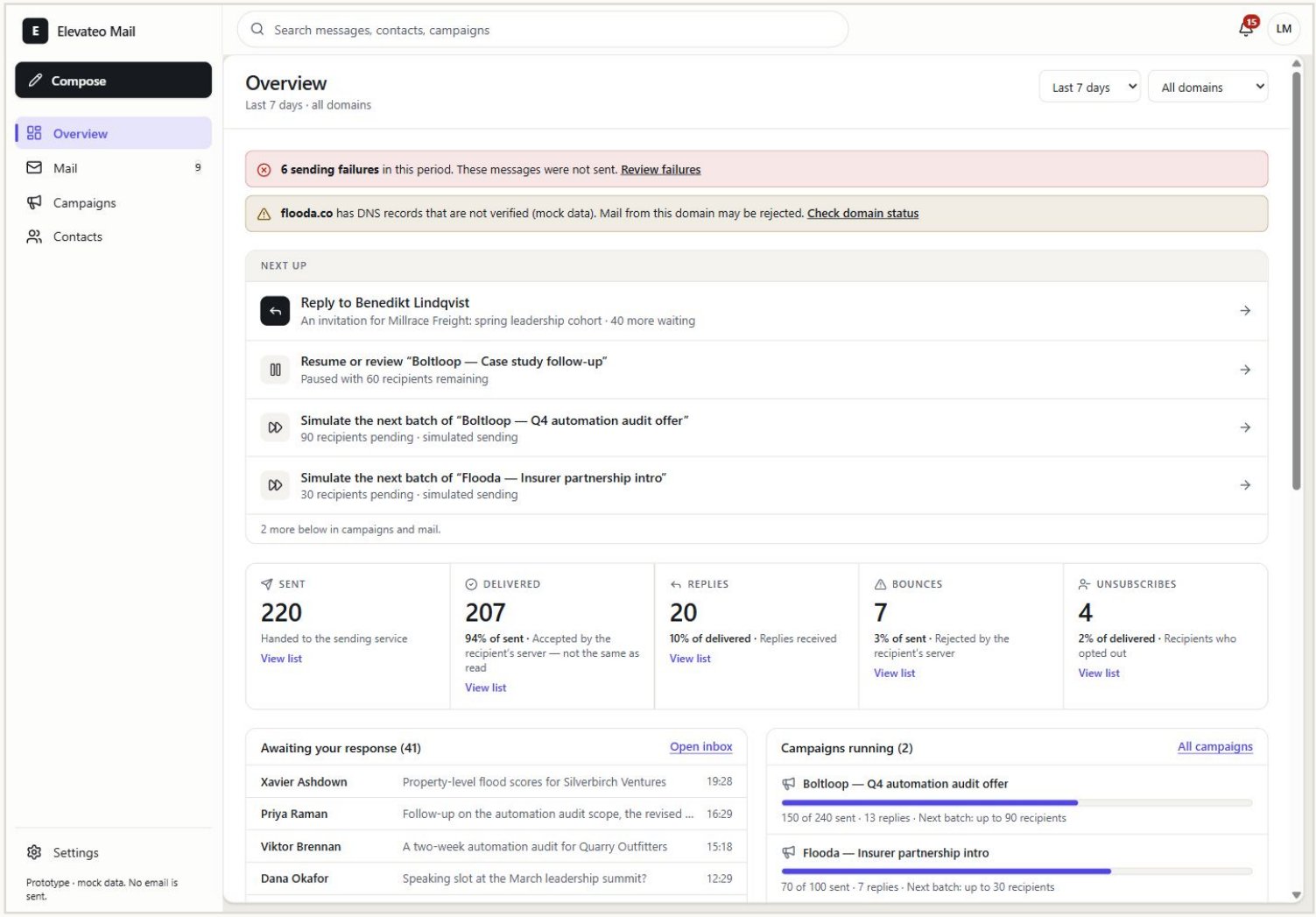
Reopen the saved board in Excalidraw. Inspect every export for readable text, intact images, clipping, correct arrows and missing states. Add a legend and written status labels. Under each diagram explain: what it does, why, and what can fail. Verify new provider claims with official sources; cite them and flag assumptions. Do not deploy, send email, change DNS or create credentials.

PROMPT END

Brief basis: approved user requirements, product master plan, backend blueprint, distribution addendum and verified Fable frontend screenshots (4 October 2026). File-format reference: official Excalidraw JSON schema. The files field holds embedded image data.

R1 / Overview

overview-light.png

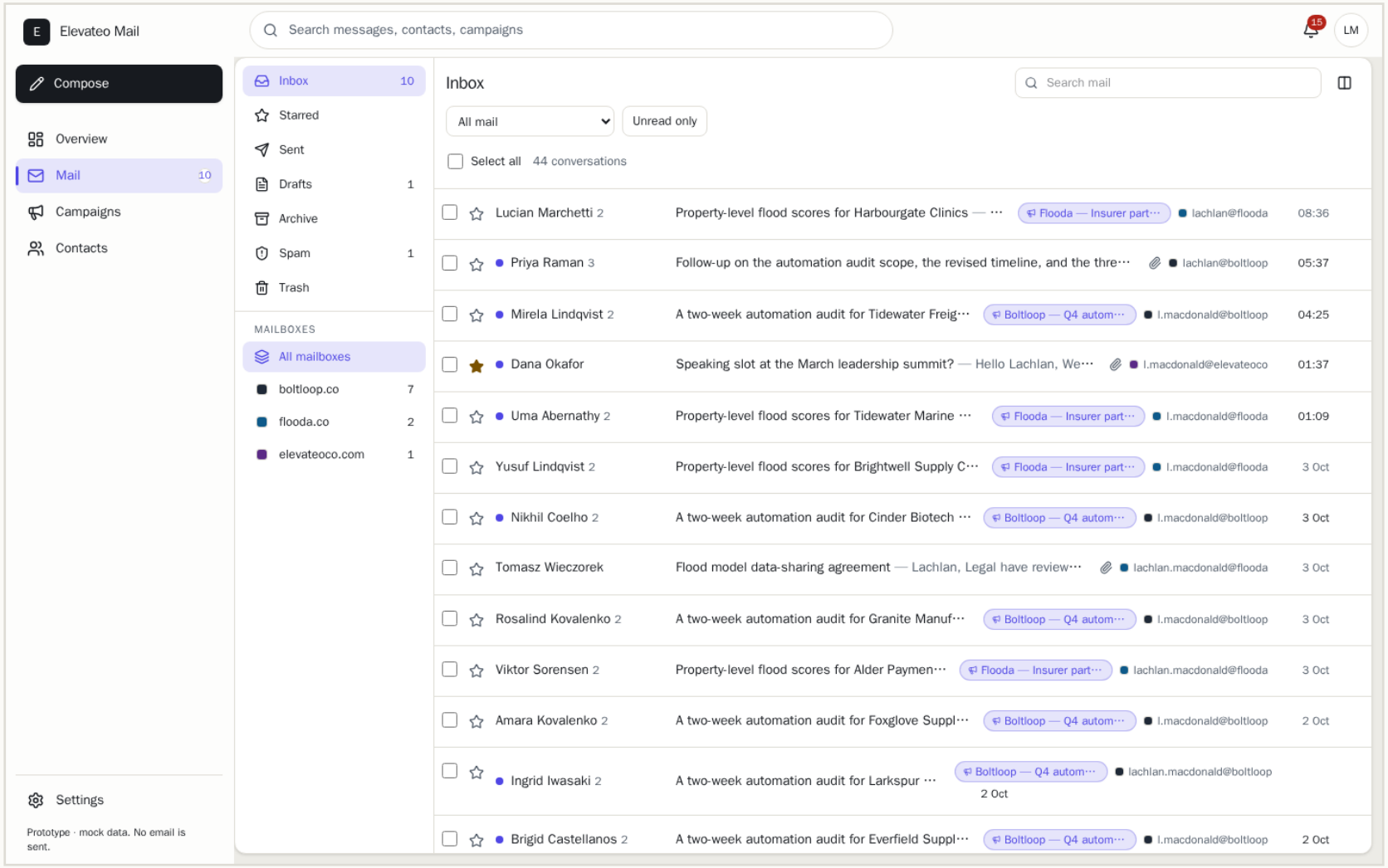


Start with work that needs attention

Preserve Next up, clear failures and figures that open useful lists. Diagram link: campaign jobs, sending results and connection health. Add proposed stale-sync and allowance states; do not treat sample figures as real throughput.

R2 / Inbox

inbox-desktop.png

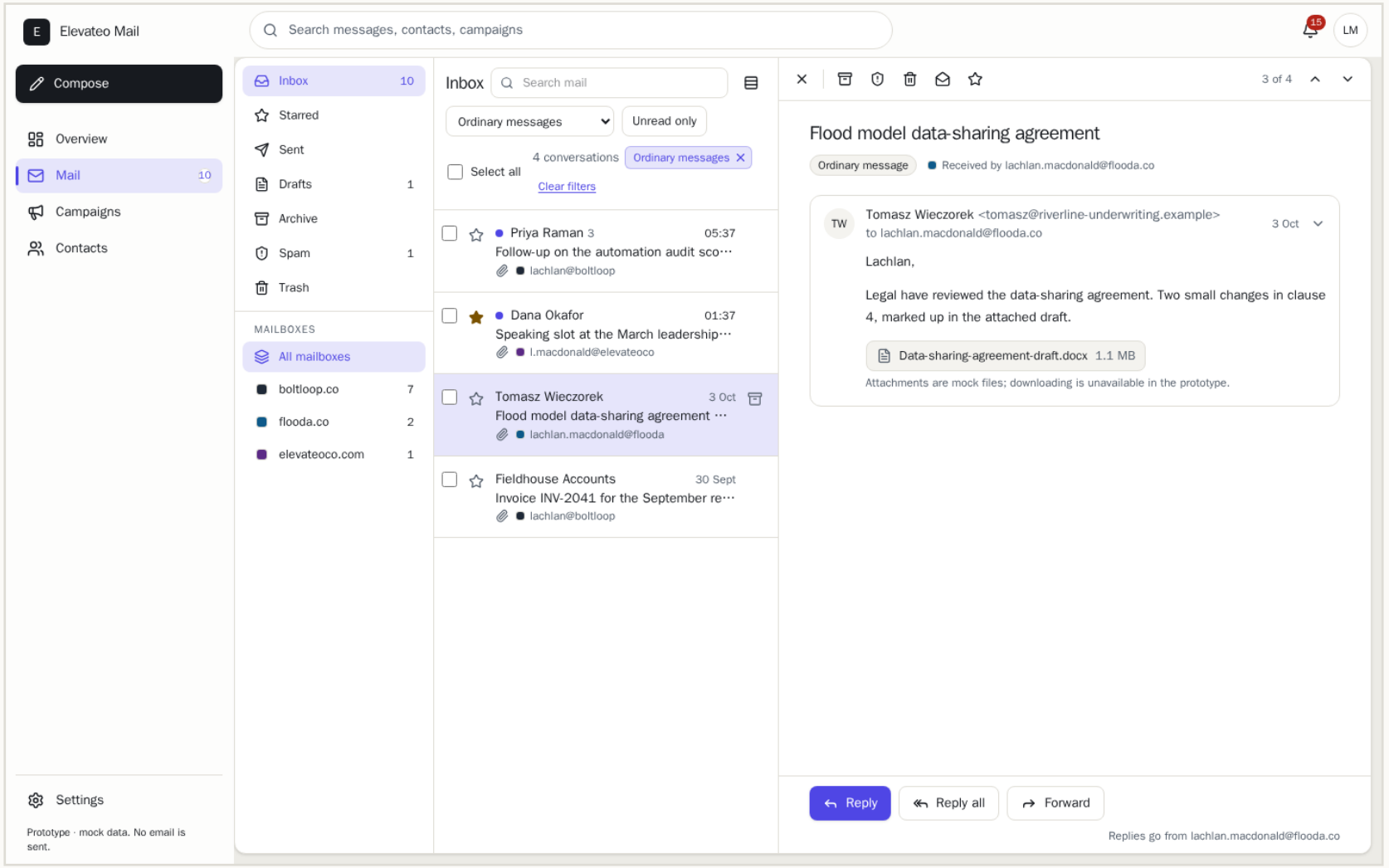


Make mailbox identity and scope unmistakable

Keep the mailbox rail, visible filters, row identity and quick actions. Diagram link: indexed message lists and per-user ownership checks. Sketch empty, loading and disconnected states beside the happy path.

R3 / Split reader

inbox-split-desktop.png

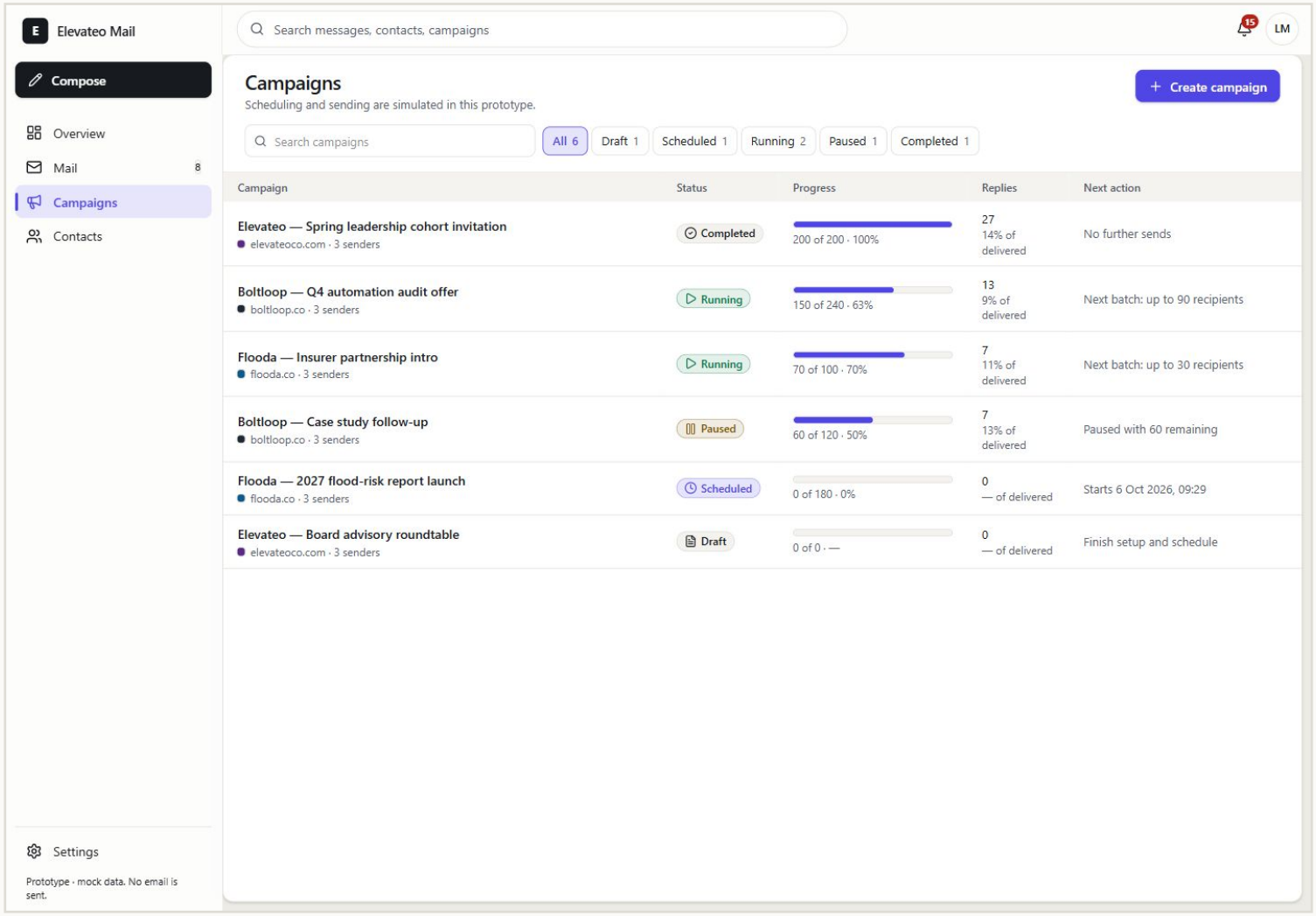


Read and reply without losing your place

Keep list context and the actual sender account visible. Diagram link: on-demand message body, draft storage and the final send gate. Add a clear quota or approval failure beside the reply action.

R4 / Campaign management

campaigns-desktop.png

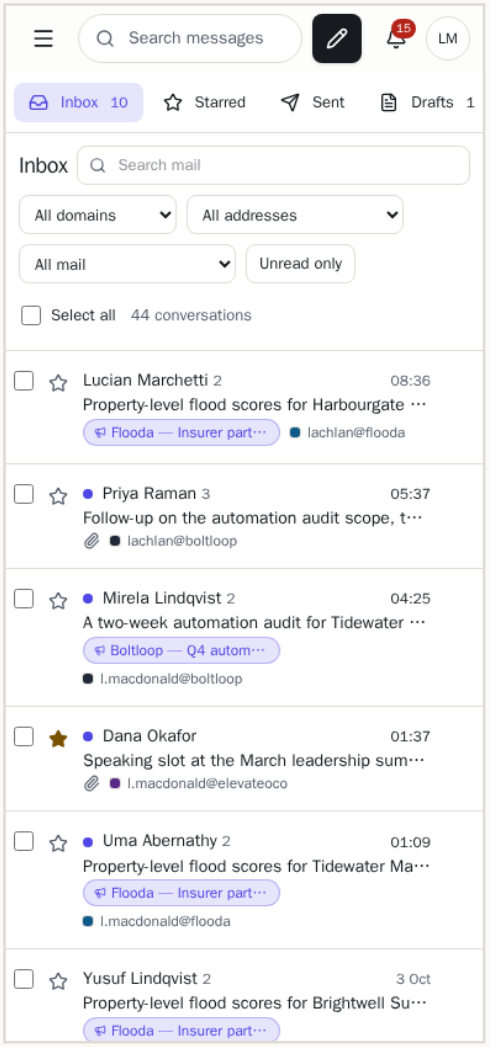


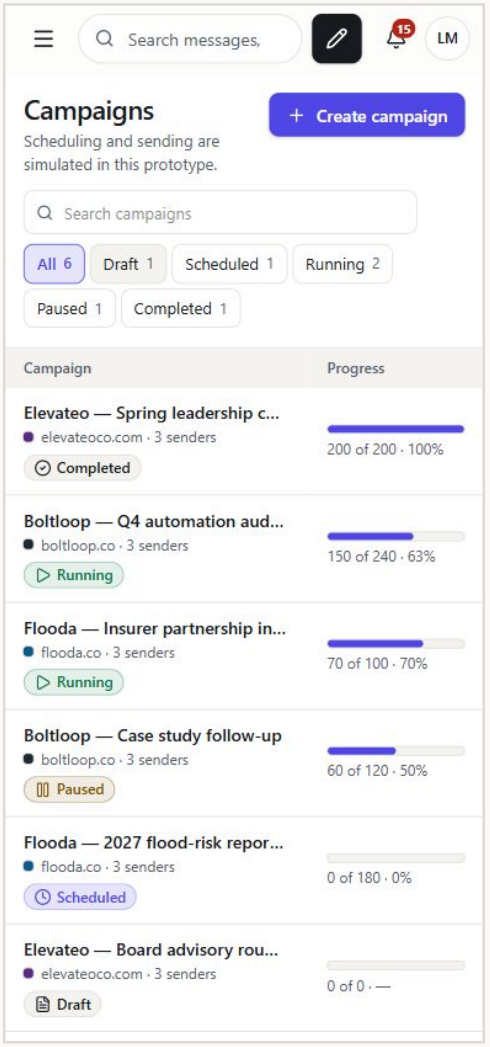
See progress, then take the next action

Preserve statuses, progress, reply figures and clear campaign entry points. Diagram link: cloud schedule, sending ledger and provider events. Sketch pause, consent checks and unknown-send recovery; delivery is not proof of reading.

R5 + R6 / Mobile workflows

inbox-mobile.png + campaigns-mobile.png



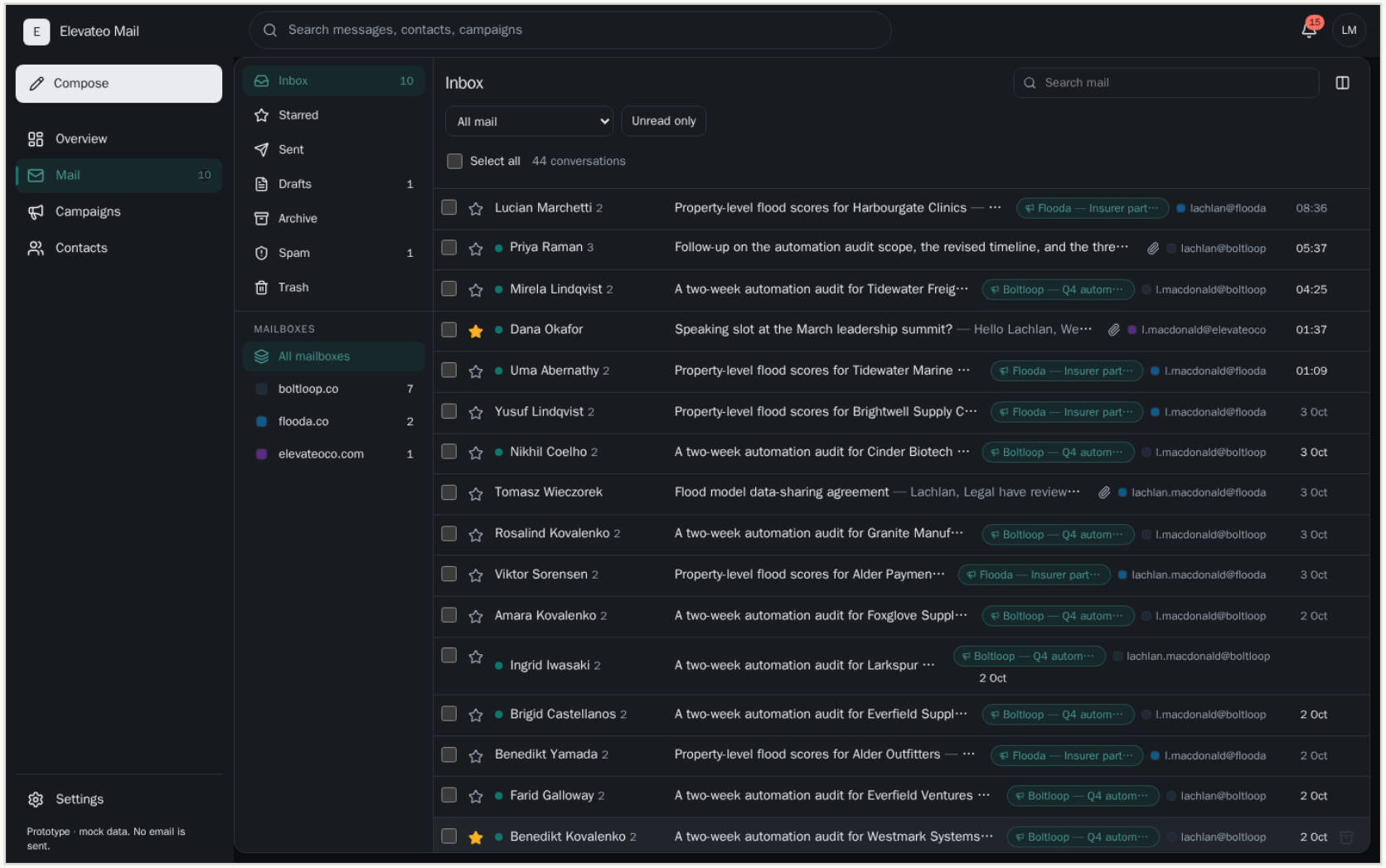


One focused task per screen

Use a drawer for navigation and a list-to-detail flow for mail. Keep filters, mailbox identity and back navigation usable. Diagram link: the same permissions and backend contracts as desktop; a smaller screen gets no weaker rules.

R7 / Dark appearance

inbox-dark.png



Keep the same hierarchy in a different theme

Preserve contrast, selected rows, readable labels and keyboard focus. Diagram link: shared UI tokens, not a second application. In proposed states, use words and icons as well as colour for errors, permissions and send status.